

.NET FULLSTACK

SENIOR LEVEL INTERVIEW PREP GUIDE

ASP.NET Core · Entity Framework Core · Angular · SQL Server · RabbitMQ · Redis · Azure · Docker · Microservices · Clean Architecture ·
SOLID · Design Patterns

30 Domains · 45+ Deep-Dive Questions · Production Code Examples · Real-World Scenarios

TABLE OF CONTENTS

1.	Microservices Architecture & Project Walkthrough
2.	Scalability, Performance & Reliability
3.	Dependency Injection – Internals & Lifetimes
4.	Microservice Communication – REST · RabbitMQ · Saga
5.	Caching – In-Memory · Redis · Response Cache
6.	API Security – JWT · OAuth · API Gateway
7.	IHostedService vs BackgroundService
8.	Entity Framework Core – Deep Performance Tuning
9.	High-Traffic API Design & Large Data Processing
1	Clean Architecture / Onion Architecture
0.	
1	Angular – Change Detection & OnPush
1.	
1	Scalable Angular Application Design
2.	
1	State Management – Service vs NgRx
3.	
1	RxJS – Subject · BehaviorSubject · ReplaySubject
4.	
1	Angular Security & Token Refresh Flow
5.	
1	Angular Performance Optimization
6.	
1	Template-driven vs Reactive Forms
7.	
1	Dynamic Forms · Infinite Scrolling · Virtual Scroll
8.	
1	Angular Lifecycle Hooks
9.	
2	Angular ↔ .NET Core Integration
0.	
2	SQL Server – Table Design & Indexing
1.	
2	Clustered vs Non-clustered Indexes
2.	
2	SQL Query Optimization – Tools & Techniques
3.	

2 Transactions & Isolation Levels

4.

2 Stored Procedures for Large Data Processing

5.

2 Pagination for Millions of Records

6.

2 Normalization vs Denormalization

7.

2 DB Versioning & Migrations in CI/CD

8.

2 Database Security

9.

3 Data Modelling for New Modules / Microservices

0.

★ BONUS – SOLID · Design Patterns · Middleware · async/await · Azure · Docker · MCQ Cheat Sheet

.

DOMAIN 1

Microservices Architecture & Project Walkthrough

Architecture · Roles · Communication · Deployment

Q1. Walk me through a microservices-based project you worked on and explain your role.

Microservices is an architectural style where a large application is broken into small, independently deployable services, each owning its own data and communicating through well-defined APIs or events. The main benefit is that teams can develop, deploy, and scale each service independently — a billing bug doesn't take down the patient portal.

PROJECT OVERVIEW

In my recent healthcare project, we migrated a monolith into **6 microservices: Identity, Patient, Billing, Appointment, Notification, and Reporting**. Each had its own SQL Server database, its own Docker container, and its own Azure DevOps CI/CD pipeline. The Angular frontend never talked directly to any service — everything went through an **API Gateway (Ocelot/YARP)** that handled authentication, rate limiting, and routing.

ARCHITECTURE DIAGRAM

```
Angular SPA
├──
└──
    API Gateway (Ocelot/YARP) ← JWT validation, rate limiting, request routing
    ├──
    │   IdentityService [SQL Server] issues JWT tokens
    │   PatientService [SQL Server] CRUD + EF Core
    │   BillingService [SQL Server] RabbitMQ publisher
    │   AppointmentService [SQL Server] event-driven
    │   NotificationService [SQL Server] RabbitMQ consumer
    │   ReportingService [Read replica] heavy queries, Redis cache
    └──
        Cross-cutting: Redis Cache | RabbitMQ | Azure Key Vault | Serilog + App Insights
        Deployment: Docker → AKS (Azure Kubernetes Service) | Azure DevOps CI/CD
```

MY ROLE – Lead .NET Full Stack Developer

- Designed Clean Architecture APIs in ASP.NET Core 6/8
- Implemented JWT authentication with refresh token rotation
- Built RabbitMQ event publishers and consumers using MassTransit
- Optimised EF Core queries (AsNoTracking, projections, bulk ops) — cut import time from 15 min → 45 sec
- Implemented Redis distributed cache — reduced DB load by 80%

- Created BackgroundService queue processors for async email/SMS
- Built lazy-loaded Angular feature modules with NgRx state management
- Set up Serilog + Application Insights with CorrelationId tracing across all services
- Mentored 3 junior developers and led code reviews

■ **Real-world scenario:** *When a patient books an appointment: AppointmentService saves the record, then publishes an 'AppointmentBooked' event to RabbitMQ. Three consumers react independently: NotificationService sends SMS, BillingService creates an invoice, AnalyticsService updates dashboards. If Notification goes down, the booking still succeeds — messages wait in the queue and are processed when it recovers. Dead-letter queues catch poison messages after 3 failed retries.*

■ **Pro Tip:** *When answering this in an interview, always mention: bounded context, database per service, how you handle distributed consistency, and your monitoring strategy. These separate mid-level from senior answers.*

DOMAIN 2

Scalability, Performance & Reliability in .NET Core APIs

Stateless APIs · async/await · Redis · Polly · Health Checks

Q2. How do you ensure scalability, performance, and reliability in .NET Core APIs?

Think of this as a three-layer problem. **Scalability** is about the architecture — can you add more instances without breaking anything? **Performance** is about code-level efficiency — are you wasting CPU cycles or database round-trips? **Reliability** is about failure tolerance — what happens when a downstream service dies or a DB query times out?

SCALABILITY

- **Stateless APIs:** No server-side session. All user context in JWT claims. Any instance can serve any request — perfect for horizontal scaling on Kubernetes.
- **async/await everywhere:** Never block a thread on I/O. A thread released back to the pool can serve another request immediately.
- **Redis distributed cache:** Shared cache across all instances. Master data, reference lookups, dashboard aggregations — avoid repeated DB hits.
- **Load balancing:** Kubernetes ingress or Azure Application Gateway distributes traffic across pods.

PERFORMANCE — CODE EXAMPLES

```
// ■ Projection + NoTracking + Pagination = fast read

var patients = await _context.Patients

.AsNoTracking() // skip change-tracker

.Where(p => p.IsActive && p.HospitalId == hospitalId)

.Select(p => new PatientDto { // only columns you need

    Id = p.Id,

    Name = p.Name,

    DOB = p.DateOfBirth

})

.Skip(page * pageSize).Take(pageSize) // server-side pagination

.ToListAsync();

// ■ Response compression (reduces payload ~60-70%)

builder.Services.AddResponseCompression(opts => {

    opts.EnableForHttps = true;

    opts.Providers.Add<BrotliCompressionProvider>();
```

```
});
```

RELIABILITY — POLLY POLICIES

```
// ■ Retry + Circuit Breaker for outbound HTTP calls

builder.Services.AddHttpClient("PaymentAPI", c => {

    c.BaseAddress = new Uri(config["PaymentAPI:BaseUrl"]);

    c.Timeout = TimeSpan.FromSeconds(5);

})

.AddTransientHttpErrorPolicy(p => // retry on 5xx/timeout

p.WaitAndRetryAsync(3, attempt =>

    TimeSpan.FromSeconds(Math.Pow(2, attempt)))) // 2s, 4s, 8s

.AddTransientHttpErrorPolicy(p => // open circuit after 5 failures

p.CircuitBreakerAsync(5, TimeSpan.FromSeconds(30)));

// ■ Global exception middleware

public async Task InvokeAsync(HttpContext ctx) {

    try { await _next(ctx); }

    catch (ValidationException ex) {

        ctx.Response.StatusCode = 400;

        await ctx.Response.WriteAsJsonAsync(new { error = ex.Message });

    }

    catch (Exception ex) {

        _logger.LogError(ex, "Unhandled exception");

        ctx.Response.StatusCode = 500;

        await ctx.Response.WriteAsJsonAsync(new { error = "Internal error" });

    }

}

// ■ Health checks
```

```
builder.Services.AddHealthChecks()  
  
    .AddSqlServer(connStr)  
  
    .AddRedis(redisConn)  
  
    .AddRabbitMQ(rabbitConn);  
  
app.MapHealthChecks("/health");
```

■ **Real-world scenario:** HCA portal during annual claim season: 10,000 requests/min. Before optimization: DB CPU at 95%, p99 latency 4 seconds. After: Redis cached master doctor/hospital data (10-min TTL) → DB hits dropped 80%. Polly circuit breaker tripped when Payment gateway slowed — fallback returned 'payment pending' instead of cascading 500 errors. p99 latency dropped to 280ms.

DOMAIN 3

Dependency Injection – Internals & Lifetimes

IoC Container · ServiceCollection · ServiceProvider · Lifetimes

Q3. What is Dependency Injection in .NET Core and how does it work under the hood?

Dependency Injection (DI) is the practice of providing a class its dependencies from the outside rather than letting it create them internally. This keeps classes loosely coupled, testable, and easy to swap. In .NET Core, DI is a first-class citizen built into the framework — you don't need a third-party container like Unity or Autofac for most scenarios.

WITHOUT DI — THE PROBLEM

```
// ■ Tight coupling – impossible to unit test without sending real emails

public class OrderService {

    private SmtplibEmailService _email = new SmtplibEmailService(); // hard dependency

    public void PlaceOrder(Order order) {

        // ... save order ...

        _email.Send(order.CustomerEmail, "Order confirmed!");

    }

}
```

WITH DI — THE SOLUTION

```
// ■ Depend on abstraction

public interface IEmailService { Task SendAsync(string to, string body); }

public class SmtplibEmailService : IEmailService {

    public Task SendAsync(string to, string body) { /* SMTP logic */ }

}

// Inject interface – OrderService has no idea which implementation it gets

public class OrderService {

    private readonly IEmailService _email;

    public OrderService(IEmailService email) => _email = email;

    public async Task PlaceOrder(Order order) {
```

```
// ... save order ...

await _email.SendAsync(order.CustomerEmail, "Order confirmed!");

}

}

// Program.cs – register once, use everywhere

builder.Services.AddScoped<IEmailService, SmtplibEmailService>();

// In tests: mock.Setup(e => e.SendAsync(...)).ReturnsAsync();
```

HOW IT WORKS UNDER THE HOOD

When you call **builder.Services.Add...()**, .NET adds a **ServiceDescriptor** to a **ServiceCollection** — basically a dictionary of (interface → implementation, lifetime). When **builder.Build()** is called, a **ServiceProvider** is created from that collection. At request time, the **ServiceProvider** resolves the object graph **recursively**: it sees **OrderService** needs **IEmailService**, creates **SmtplibEmailService** (respecting lifetime), injects it, then returns **OrderService**.

Lifetime	Instances Created	Disposed	When to Use	Example
Singleton	Once at first resolve	App shutdown	Thread-safe, shared state	Redis connection, config, logging
Scoped	Once per HTTP request	End of request	Per-request state	DbContext, repositories, UoW
Transient	Every time resolved	After use	Lightweight, stateless	Validators, formatters, utilities

■ **Common Mistake:** Never inject a **Scoped** or **Transient** service into a **Singleton** — the **Singleton** will hold a stale, disposed reference. .NET will throw at startup if you enable scope validation: **builder.Services.BuildServiceProvider(validateScopes: true)**.

■ **Real-world scenario:** In **BillingService** we had **SmtplibEmailService** for dev and **SendGridEmailService** for production. We registered based on environment: *if (isProd) services.AddScoped() else services.AddScoped()*. Zero changes in **OrderService** or any controller. Unit tests injected **Mock** — no emails were ever sent during CI.

DOMAIN 4

Microservice Communication – REST · RabbitMQ · Saga

Sync vs Async · Event-driven · Distributed Transactions

Q4. How do you handle communication between microservices?

There's no one-size-fits-all answer. The choice between synchronous REST and asynchronous messaging is driven by the business requirement. Ask: **Does the caller need an immediate answer?** If yes, use REST. If the work can happen in the background, use messaging. Mixing both is common and correct.

Type	Protocol	When to Use	Trade-off
Synchronous	REST / gRPC	Need immediate response (payment check, auth)	Tight coupling; cascading failures
Asynchronous	RabbitMQ / Azure Service Bus	Fire-and-forget (email, audit, inventory)	Eventual consistency; harder to debug
Hybrid	REST + Events	Order placed (REST) → Notify (event)	Best of both worlds

RABBITMQ IMPLEMENTATION (MassTransit)

```
// Publisher – AppointmentService

public class AppointmentService {

    private readonly IPublishEndpoint _bus;

    public AppointmentService(IPublishEndpoint bus) => _bus = bus;

    public async Task<AppointmentDto> BookAsync(BookingRequest req) {

        var appt = await _repo.CreateAsync(req);

        // Publish event – don't care who listens

        await _bus.Publish(new AppointmentBookedEvent {

            AppointmentId = appt.Id,

            PatientId = appt.PatientId,

            DoctorId = appt.DoctorId,

            SlotTime = appt.SlotTime,

        });

    }

}
```

```

return appt;

}

}

// Consumer - NotificationService

public class AppointmentBookedConsumer : IConsumer<AppointmentBookedEvent> {

    private readonly ISmsService _sms;

    public AppointmentBookedConsumer(ISmsService sms) => _sms = sms;

    public async Task Consume(ConsumeContext<AppointmentBookedEvent> ctx) {

        var msg = ctx.Message;

        await _sms.SendAsync(msg.PatientPhone,

            $"Appointment confirmed for {msg.SlotTime:dd MMM HH:mm}");

    }

}

// MassTransit registration with retry + dead-letter

builder.Services.AddMassTransit(x => {

    x.AddConsumer<AppointmentBookedConsumer>();

    x.UsingRabbitMq((ctx, cfg) => {

        cfg.Host("rabbitmq://localhost");

        cfg.ReceiveEndpoint("notification-queue", e => {

            e.UseMessageRetry(r => r.Intervals(1000, 5000, 10000)); // 3 retries

            e.ConfigureConsumer<AppointmentBookedConsumer>(ctx);

        });

    });

});

});

});

```

SAGA PATTERN — DISTRIBUTED TRANSACTIONS

When a business operation spans multiple services, you can't use a traditional database transaction. The Saga pattern sequences steps with compensating actions on failure.

```
// Order placement saga: Order → Payment → Inventory → Notification

//

// Happy path: OrderCreated → PaymentProcessed → StockReduced → EmailSent

// On failure: PaymentFailed → OrderCancelled (compensation)

// StockInsufficient → PaymentRefunded → OrderCancelled


public class OrderSaga : MassTransitStateMachine<OrderSagaState> {

    public OrderSaga() {

        Initially(

            When(OrderSubmitted)

                .Then(ctx => ctx.Saga.OrderId = ctx.Message.OrderId)

                .PublishAsync(ctx => ctx.Init<ProcessPaymentCommand>(new {

                    ctx.Message.OrderId, ctx.Message.Amount

                })))

            .TransitionTo(AwaitingPayment));

        During(AwaitingPayment,

            When(PaymentProcessed)

                .PublishAsync(ctx => ctx.Init<ReduceStockCommand>(new { ctx.Message.OrderId })))

            .TransitionTo(AwaitingStock),

            When(PaymentFailed)

                .PublishAsync(ctx => ctx.Init<CancelOrderCommand>(new { ctx.Message.OrderId })))

            .TransitionTo(Cancelled));

    }

}
```

■ **Real-world scenario:** E-commerce drug order: OrderService creates order (REST returns 202 Accepted). Saga orchestrates: PaymentService charges card → InventoryService reduces stock. On insufficient stock: compensating transaction refunds the payment automatically. Customer sees 'Payment refunded, item out of stock' within seconds. No manual intervention.

DOMAIN 5

Caching – In-Memory · Redis · Response Cache

Cache-aside Pattern · TTL · Invalidation · Why Redis

Q5. What caching mechanisms have you implemented and why?

Caching is one of the highest-ROI performance optimizations you can make. The key insight is: **don't compute or fetch what you already know**. The right cache layer depends on your deployment topology. In-memory is fast but only works for a single server. Redis is the production standard because it's shared across all instances, extremely fast (sub-millisecond), and supports TTL, pub/sub, and atomic operations.

Cache Type	Scope	Speed	Best For	Limitation
IMemoryCache	Single process	Fastest	Dev/single server, tiny data	Doesn't scale horizontally
IDistributedCache (Redis)	All instances	~1ms	Production multi-pod APIs	Network hop overhead
[ResponseCache] / CDN	HTTP level	Fastest for GETs	Static or slowly-changing GET responses	GET only, auth headers skip it

REDIS — CACHE-ASIDE PATTERN (PRODUCTION IMPLEMENTATION)

```
// Setup

builder.Services.AddStackExchangeRedisCache(o => {

    o.Configuration = builder.Configuration["Redis:ConnectionString"];

    o.InstanceName = "hca: "; // namespace prefix

});

// Generic cache service – reuse across the entire app

public class CacheService {

    private readonly IDistributedCache _cache;

    private readonly ILogger _logger;

    public async Task<T?> GetOrSetAsync<T>(string key, Func<Task<T>> factory,

        TimeSpan? ttl = null) {

        var cached = await _cache.GetStringAsync(key);

        if (cached != null) {
```

```

_logger.LogDebug("Cache HIT: {Key}", key);

return JsonSerializer.Deserialize<T>(cached);

}

_logger.LogDebug("Cache MISS: {Key}", key);

var value = await factory(); // fetch from DB / API

if (value != null) {

    await _cache.SetStringAsync(key,

        JsonSerializer.Serialize(value),

        new DistributedCacheEntryOptions {

            AbsoluteExpirationRelativeToNow = ttl ?? TimeSpan.FromMinutes(10)

        });

}

return value;

}

public Task InvalidateAsync(string key) => _cache.RemoveAsync(key);

}

// Usage in PatientService

public async Task<PatientDto?> GetByIdAsync(int id) {

    return await _cache.GetOrSetAsync(

        key: $"patient:{id}",

        factory: () => _repo.GetPatientDtoAsync(id),

        ttl: TimeSpan.FromMinutes(5));

}

// Invalidate on update

```



```
public async Task UpdateAsync(Patient patient) {  
  
    await _repo.UpdateAsync(patient);  
  
    await _cache.InvalidateAsync($"patient:{patient.Id}");  
  
}
```

■ **Real-world scenario:** Doctor lookup API hit 2,000 req/min. DB CPU was at 90%, costing extra Azure SQL DTUs. Moved master doctor/hospital lookup to Redis with 10-minute TTL. Cache hit ratio reached 94%. DB CPU dropped to 15%. Monthly Azure SQL cost dropped by \$400. Cache invalidation triggered on admin update — no stale data issues.

■ **Pro Tip:** Cache keys should be namespaced: 'service:entity:id' e.g. 'patient:42'. Always set a TTL — never cache forever. Design for cache miss gracefully.

API Security – JWT · OAuth · API Gateway

Token Generation · Validation · Refresh Flow · Key Vault

Q6. How do you secure .NET Core APIs using JWT? Walk through the full flow.

JWT (JSON Web Token) is a stateless authentication mechanism. The server issues a signed token containing claims (user ID, roles, permissions). Every subsequent request includes this token in the Authorization header. The server validates the signature — no DB lookup needed per request. This is what makes JWT ideal for stateless, scalable microservices.

FULL JWT FLOW

- 1. User POSTs username/password to **/api/auth/login**
- 2. API validates credentials against DB
- 3. API generates a short-lived **Access Token** (15 min) + long-lived **Refresh Token** (7 days)
- 4. Access Token returned in response body; Refresh Token in **HttpOnly cookie** (XSS-safe)
- 5. Angular stores Access Token in memory (not localStorage — XSS risk)
- 6. HTTP Interceptor adds **Authorization: Bearer <token>** to every request
- 7. API Gateway validates JWT signature before forwarding to microservices
- 8. On 401, interceptor calls **/api/auth/refresh** silently and retries

TOKEN GENERATION

```
// AuthController.cs

[HttpPost("login")]

public async Task<IActionResult> Login(LoginDto dto) {

    var user = await _userService.ValidateAsync(dto.Username, dto.Password);

    if (user == null) return Unauthorized(new { message = "Invalid credentials" });

    var claims = new[] {

        new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),

        new Claim(ClaimTypes.Name, user.Username),

        new Claim(ClaimTypes.Role, user.Role),

        new Claim("HospitalId", user.HospitalId.ToString()),

    };

    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]));

    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
```

```

var token = new JwtSecurityToken(

    issuer: _config["Jwt:Issuer"],

    audience: _config["Jwt:Audience"],

    claims: claims,

    expires: DateTime.UtcNow.AddMinutes(15),

    signingCredentials: creds);

var refreshToken = GenerateSecureRefreshToken(); // Guid.NewGuid or CSPRNG

await _tokenRepo.StoreRefreshTokenAsync(user.Id, refreshToken);

Response.Cookies.Append("refreshToken", refreshToken, new CookieOptions {

    HttpOnly = true, Secure = true, SameSite = SameSiteMode.Strict,

    Expires = DateTimeOffset.UtcNow.AddDays(7)

});

return Ok(new { accessToken = new JwtSecurityTokenHandler().WriteToken(token) });

}

```

JWT VALIDATION SETUP

```

// Program.cs

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)

.AddJwtBearer(options => {

    options.TokenValidationParameters = new TokenValidationParameters {

        ValidateIssuerSigningKey = true,

        IssuerSigningKey = new SymmetricSecurityKey(

            Encoding.UTF8.GetBytes(config["Jwt:Key"])),

        ValidateIssuer = true, ValidIssuer = config["Jwt:Issuer"],

        ValidateAudience = true, ValidAudience = config["Jwt:Audience"],

        ClockSkew = TimeSpan.Zero // no grace period

    };

```

```
});

// Protect endpoints

[Authorize(Roles = "Doctor,Admin")]

[HttpGet("patients/{id}")]

public async Task<IActionResult> GetPatient(int id) {

    var hospitalId = int.Parse(User.FindFirst("HospitalId")!.Value);

    // ... only return patients of this doctor's hospital

}
```

■ **Real-world scenario:** HCA Doctor Portal: Doctor's 15-min access token expires mid-session while writing notes. Angular interceptor silently catches the 401, calls /refresh with the HttpOnly cookie, gets a new access token, and retries the original request. Doctor never sees a login popup or loses their work. Refresh tokens are single-use and rotated on every use (refresh token rotation).

■ **Common Mistake:** Never store JWT secrets in appsettings.json in production. Use Azure Key Vault. Never store Access Tokens in localStorage — vulnerable to XSS. Use memory or sessionStorage with caution.

IHostedService vs BackgroundService

Long-running Tasks · Queue Processors · Graceful Shutdown

Q7. What is the difference between IHostedService and BackgroundService?

Both are mechanisms for running code in the background within an ASP.NET Core process. **IHostedService** is the raw interface — you control everything. **BackgroundService** is an abstract class that implements IHostedService and gives you a cleaner **ExecuteAsync** method with Cancellation token. 95% of the time you should use BackgroundService.

	IHostedService	BackgroundService
Type	Interface	Abstract class implements IHostedService
Methods to implement	StartAsync + StopAsync (both manually)	Just override ExecuteAsync
Loop control	You manage the loop	Loop implied — runs until token cancelled
Graceful shutdown	Manual stop logic	Cancellation token handles it
Use for	Precise lifecycle (startup tasks)	Long-running queues, schedulers, pollers

```
// ■ Real-world BackgroundService: Email queue processor

public class EmailQueueWorker : BackgroundService {

    private readonly IServiceProvider _sp; // use factory for scoped services

    private readonly ILogger<EmailQueueWorker> _logger;

    public EmailQueueWorker(IServiceProvider sp, ILogger<EmailQueueWorker> logger) {

        _sp = sp; _logger = logger;

    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken) {

        _logger.LogInformation("Email worker started");

        while (!stoppingToken.IsCancellationRequested) {

            try {

                // MUST create scope – DbContext is Scoped, worker is Singleton

                using var scope = _sp.CreateScope();

                var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
```

```

var email = scope.ServiceProvider.GetRequiredService<IEmailService>();

var pending = await db.EmailQueue

.Where(e => !e.IsSent && e.RetryCount < 3)

.OrderBy(e => e.CreatedAt)

.FirstOrDefaultAsync(stoppingToken);

if (pending != null) {

    await email.SendAsync(pending.To, pending.Subject, pending.Body);

    pending.IsSent = true;

    pending.SentAt = DateTime.UtcNow;

    await db.SaveChangesAsync(stoppingToken);

    _logger.LogInformation("Email sent to {To}", pending.To);

} else {

    await Task.Delay(TimeSpan.FromSeconds(10), stoppingToken);

}

} catch (OperationCanceledException) { break; }

catch (Exception ex) {

    _logger.LogError(ex, "Error processing email queue");

    await Task.Delay(TimeSpan.FromSeconds(30), stoppingToken);

}

}

_logger.LogInformation("Email worker stopped gracefully");

}

}

// Register in Program.cs

builder.Services.AddHostedService<EmailQueueWorker>();

```

■ **Pro Tip:** Always inject *IServiceProvider* (not *DbContext* directly) into *BackgroundService*. *BackgroundService* is a *Singleton* — injecting a *Scoped DbContext* directly will cause a captive dependency error at runtime.

■ **Real-world scenario:** *HCA NotificationService*: *RabbitMQ* consumer + email sender ran as *BackgroundService* in *K8s*. When a pod receives *SIGTERM* (rolling deploy), the *CancellationToken* is triggered. *ExecuteAsync* drains the current message, then exits. No emails lost. Old *Windows Service* approach had no graceful shutdown — restarts dropped messages.

DOMAIN 8

Entity Framework Core – Deep Performance Tuning

AsNoTracking · Projections · N+1 · Bulk Ops · Compiled Queries

Q8. How do you optimise Entity Framework Core for high-load production systems?

EF Core is incredibly convenient but can generate terrible SQL if you don't know what you're doing. The golden rule: **always look at the SQL EF generates** (use LogTo or a profiler). Most performance problems fall into four categories: unnecessary data loading, N+1 queries, missing indexes, and using EF for bulk operations.

Technique	Why It Helps	When to Use
AsNoTracking()	Skips change-tracker — less memory, faster reads	All read-only queries
Select projections	Fetches only needed columns → smaller payloads	List/grid APIs
Include() eager load	Avoids N+1 queries	Related entity always needed
Split queries	Avoids cartesian explosion on multi-include	Complex object graphs
Compiled queries	Pre-compiled LINQ → saves expression tree parse	Hot, repeated queries
BulkInsert/Update	100k rows in seconds vs minutes	Import jobs, migrations
Pagination	Prevents loading entire table	All list endpoints

PROJECTION + NOTRACKING (Most Important)

```
// ■ Bad – loads all columns, tracks all entities

var patients = await _context.Patients.ToListAsync();


// ■ Good – only what the UI needs, no tracking overhead

var patients = await _context.Patients

.AsNoTracking()

.Where(p => p.HospitalId == hospitalId && p.IsActive)

.OrderByDescending(p => p.LastVisitDate)

.Select(p => new PatientListDto {

    Id = p.Id,

    FullName = p.FirstName + " " + p.LastName,

    DateOfBirth = p.DateOfBirth,
```



```

    LastVisitDate = p.LastVisitDate,

    DoctorName = p.AssignedDoctor.FullName // auto-joined

})

.Skip((page - 1) * pageSize)

.Take(pageSize)

.ToListAsync();

```

N+1 QUERY PROBLEM — MOST COMMON EF MISTAKE

```

// ■ N+1: 1 query for orders + 1 query per order for customer = 1001 queries!

var orders = await _context.Orders.ToListAsync();

foreach (var order in orders)

    Console.WriteLine(order.Customer.Name); // lazy load fires each time

// ■ Fix: eager load with Include – 1 SQL JOIN = 1 round-trip

var orders = await _context.Orders

    .AsNoTracking()

    .Include(o => o.Customer)

    .Include(o => o.Items).ThenInclude(i => i.Product)

    .ToListAsync();

// ■ Alternative: Split queries (avoids cartesian product with multiple Includes)

var orders = await _context.Orders

    .AsNoTracking()

    .Include(o => o.Items).ThenInclude(i => i.Product)

    .AsSplitQuery() // runs as 2 separate queries, not 1 massive JOIN

    .ToListAsync();

```

BULK OPERATIONS (EFCore.BulkExtensions)

```

// ■ Import 80,000 claims – normal SaveChanges = timeout

// BulkInsert = 45 seconds

```

```

await _context.BulkInsertAsync(claims, options => {

options.BatchSize = 5000;

options.SetOutputIdentity = false;

options.PreserveInsertOrder = false;

});

// ■ Bulk update – update status for 50k records

await _context.BulkUpdateAsync(claimsToUpdate);

```

HOW TO MONITOR EF SQL

```

// Log all SQL in development

builder.Services.AddDbContext<AppDbContext>(o =>

o.UseSqlServer(connStr)

.LogTo(Console.WriteLine, LogLevel.Information)

.EnableSensitiveDataLogging() // dev only!

);

```

■ **Real-world scenario:** HCA claims import: 80,000 claims/day. Normal `AddRange` + `SaveChanges` timed out at 15 minutes. Switched to `BulkInsertAsync` with batch size 5,000 → 45 seconds. Also found an N+1 in the patient dashboard: 500 patients × 1 query each for doctor name = 500 extra DB round-trips per page load. Fixed with `Include` → page load from 8s to 400ms.

DOMAIN 9

High-Traffic API Design & Large Data Processing

Pagination · Queue-based Processing · Rate Limiting · Streaming

Q9. How do you design APIs to handle high traffic and large data volumes?

The core principle is: **never make the user wait for work that can happen asynchronously**. For large data, never load everything into memory at once. For high traffic, protect the system with rate limiting and offload computation to background workers.

PAGINATION — SERVER SIDE ALWAYS

```
[HttpGet]

public async Task<IActionResult> GetClaims([FromQuery] int page = 1,

[FromQuery] int pageSize = 50) {

    pageSize = Math.Min(pageSize, 200); // cap at 200 to prevent abuse

    var total = await _context.Claims.CountAsync();

    var claims = await _context.Claims

        .AsNoTracking()

        .OrderByDescending(c => c.CreatedAt)

        .Skip((page - 1) * pageSize)

        .Take(pageSize)

        .Select(c => new ClaimDto { ... })

        .ToListAsync();

    return Ok(new PagedResult<ClaimDto> {

        Data = claims,

        Page = page,

        PageSize = pageSize,

        TotalCount = total,

        TotalPages = (int)Math.Ceiling(total / (double)pageSize)

    });
}
```

```
}
```

QUEUE-BASED PROCESSING — RESPOND IMMEDIATELY, PROCESS LATER

```
// ■ Heavy report: accept and enqueue, don't block

[HttpPost("reports/monthly")]

public async Task<IActionResult> GenerateMonthlyReport([FromBody] ReportRequest req) {

    var jobId = Guid.NewGuid();

    await _bus.Publish(new GenerateReportCommand {

        JobId = jobId,

        Month = req.Month,

        HospitalId = req.HospitalId,

        RequestedBy = User.Identity!.Name

    });

    return Accepted(new { jobId, statusUrl = $"/reports/status/{jobId}" });

    // Client polls /reports/status/{jobId} or receives SignalR notification

}
```

RATE LIMITING (.NET 7+)

```
builder.Services.AddRateLimiter(opts => {

    opts.AddFixedWindowLimiter("api", o => {

        o.PermitLimit = 100;

        o.Window = TimeSpan.FromSeconds(10);

        o.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;

        o.QueueLimit = 5;

    });

    opts.RejectionStatusCode = 429;

});

app.UseRateLimiter();
```

```
[EnableRateLimiting("api")]
```

```
[HttpGet("patients")] public IActionResult GetPatients() { ... }
```

■ **Real-world scenario:** HCA monthly PDF report: 2M rows, takes 4 minutes to generate. Old approach: API blocked thread for 4 min, browser timed out, user refreshed, 5 more requests queued. New approach: POST returns 202 with jobId in 50ms. BackgroundService generates PDF, uploads to Azure Blob, pushes SignalR notification. User gets a download link. Server handles 5 concurrent reports without issue.

Clean Architecture / Onion Architecture

Q10. Explain Clean Architecture and how you applied it in a real project.

Layer	Lives In	Contains	Depends On
Domain	HCA.Domain	Entities, Value Objects, Domain Events, IRepository interfaces	Nothing — zero external dependencies
Application	HCA.Application	CQRS Commands/Queries, MediatR handlers, DTOs, validators, IEmailService interface	Domain only
Infrastructure	HCA.Infrastructure	EF Core DbContext, Repository implementations, SMTP, RabbitMQ, Redis clients	Application + Domain
API / Presentation	HCA.API	ASP.NET Controllers, DI wiring, Middleware, Program.cs	All layers

.NET Fullstack Senior Interview Prep Guide

[illegible]

■ **Real-world scenario:** Started with EF Core and business logic in controllers — unit testing required a real DB. Refactored to Clean Architecture. Domain + Application layers: 95% unit test coverage with zero DB dependency. When the client asked to switch from SQL Server to PostgreSQL for one service, we changed only Infrastructure — zero business logic changes. Build time for unit tests: 8 seconds.

■ **Pro Tip:** CQRS (Command Query Responsibility Segregation) pairs perfectly with Clean Architecture. Commands change state (return void/id). Queries return data (no side effects). MediatR dispatches both through a clean pipeline of behaviors (logging, validation, caching).

DOMAIN 11

Angular – Change Detection & OnPush Strategy

Default vs OnPush · Zones · ChangeDetectorRef

Q11. Explain Change Detection in Angular. When and why do you use OnPush?

Angular's change detection decides when to update the DOM. By default it checks the entire component tree on every browser event, HTTP response, or timer. OnPush tells Angular: 'only check this component when something I explicitly care about changes.' This can dramatically reduce DOM operations in complex UIs.

Strategy	Checks When	Performance	Safe For
Default	Every zone event (click, timer, HTTP, setTimeout)	O(n) full tree	Simple apps
OnPush	@Input reference changes · Component event · async pipe emits · markForCheck() called	Skips untouched subtrees	Data grids, dashboards, read-heavy components

```
@Component({
  selector: 'app-patient-grid',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <tr *ngFor="let p of patients; trackBy: trackById">
      <td>{{ p.name }}</td>
      <td>{{ p.lastVisit | date }}</td>
    </tr>
  `
})
export class PatientGridComponent {
  @Input() patients: Patient[] = []; // MUST pass new array ref for OnPush to trigger

  // If pushing data imperatively (not via @Input), call markForCheck()

  constructor(private cdr: ChangeDetectorRef) {}

  loadFromSocket(data: Patient[]) {
```

```

this.patients = data;

this.cdr.markForCheck(); // tells Angular: re-check this subtree

}

trackById = (_, number, p: Patient) => p.id;

}

```

■ **Real-world scenario:** Patient dashboard grid: 5,000 rows, filtered live as user types. Default CD caused 200ms freeze per keystroke because Angular checked ALL components on every keyup. Switched grid to OnPush + trackBy: freeze disappeared. FPS went from 12 to 60.

DOMAIN 12

Scalable Angular Application Design

Module Structure · Lazy Loading · Core/Shared Pattern

Q12. How do you structure and scale large Angular applications?

The key insight is: Angular apps become slow to load and hard to maintain when everything is in one module. The solution is a deliberate module strategy — Core for singletons, Shared for reusable components, Feature modules loaded lazily on demand.

```

// Recommended folder structure

src/app/

core/ // Singleton services, guards, interceptors – loaded once

services/ // AuthService, CacheService, LoggerService

guards/ // AuthGuard, RoleGuard

interceptors/ // AuthInterceptor, ErrorInterceptor

core.module.ts // forRoot() pattern to prevent double-instantiation

shared/ // Dumb components, pipes, directives – no business logic

components/ // DataTableComponent, LoadingSpinnerComponent

pipes/ // DateFormatPipe, CurrencyInrPipe

shared.module.ts

```

```
features/ // Lazy-loaded domain modules

patient/ // PatientModule – loaded only when user visits /patients

billing/ // BillingModule

reports/ // ReportsModule


// app-routing.module.ts – lazy loading

const routes: Routes = [

  { path: 'patients', loadChildren: () =>

    import('./features/patient/patient.module').then(m => m.PatientModule) },

  { path: 'billing', loadChildren: () =>

    import('./features/billing/billing.module').then(m => m.BillingModule) },

];
```

■ **Real-world scenario:** Initial bundle was 4.2 MB — 8-second load on 3G. After lazy loading all feature modules and moving third-party chart library to dynamic import within the Reports module: initial bundle dropped to 890 KB. Reports page loaded charts only when navigated to.

Approach	When	Pros	Cons
BehaviorSubject Service	Small-medium apps, 1-3 components sharing state	Simple, no boilerplate	Harder to debug at scale
NgRx Store	Large enterprise, complex async flows, team > 3	Predictable, DevTools, time-travel debug	Steep learning curve, verbose

```
@Injectable({ providedIn: 'root' })

export class AuthStateService {

  private user$ = new BehaviorSubject<User | null>(null);

  currentUser$ = this.user$.asObservable(); // expose read-only

  login(user: User) { this.user$.next(user); }

  logout() { this.user$.next(null); }

  get currentUser() { return this.user$.value; } // sync access

}
```

```
// Actions

export const loadPatients = createAction('[Patient] Load', props<{page:number}>());

export const loadPatientsSuccess = createAction('[Patient] Success',

props<{ patients: Patient[]; total: number }>());

// Reducer

const patientReducer = createReducer(initialState,

on(loadPatientsSuccess, (state, { patients, total }) =>

({ ...state, patients, total, loading: false })));
```

```
// Effect

@Injectable()

export class PatientEffects {

  load$ = createEffect(() => this._actions$.pipe(

    ofType(loadPatients),

    switchMap(({ page }) => this._api.getPatients(page).pipe(

      map(res => loadPatientsSuccess(res)),

      catchError(err => of(loadPatientsFailed({ error: err.message }))))

  ));

}
```

Q14. What is the difference between Subject, BehaviorSubject, and ReplaySubject?

Type	Has Initial Value	Late Subscriber Gets	Buffer Size	Use Case
Subject	No	Nothing (misses past)	0	Event bus, click streams, one-shot events
BehaviorSubject	Yes (required)	Current/latest value	1	Shared UI state, current user, form state
ReplaySubject(n)	No	Last n values	n (configurable)	Chat history, recent errors, audit log
AsyncSubject	No	Last value on complete	1 (on complete)	Single async result (like Promise)

■ **Real-world scenario:** *BehaviorSubject*: `currentUser$` shared across Header (avatar), Sidebar (menu items), and AuthGuard. Any component that subscribes late immediately gets the current user. *ReplaySubject(1)*: used for the 'last API error' observable in an error-log panel that loads asynchronously — it still sees the most recent error even if it subscribed after.

Angular Security & Performance

JWT Interceptors · Route Guards · trackBy · Lazy Loading · Debounce

Q15. How do you secure Angular apps and handle token expiration silently?

```
@Injectable()

export class AuthInterceptor implements HttpInterceptor {

  private isRefreshing = false;

  private refreshSubject = new BehaviorSubject<string | null>(null);

  constructor(private auth: AuthService) {}

  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {

    // Skip auth header for login/refresh endpoints

    if (req.url.includes('/auth/')) return next.handle(req);

    req = this.addToken(req, this.auth.accessToken);

    return next.handle(req).pipe(

      catchError(err => {

        if (err.status === 401 && !req.url.includes('/auth/refresh'))

          return this.handle401(req, next);

        return throwError(() => err);

      })

    );

  }

  private handle401(req: HttpRequest<any>, next: HttpHandler) {

    if (this.isRefreshing) {

      // Queue requests while refresh is in flight
```

```

return this.refreshSubject.pipe(

filter(token => token !== null), take(1),

switchMap(token => next.handle(this.addToken(req, token!))));

}

this.isRefreshing = true;

this.refreshSubject.next(null);

return this.auth.refreshToken().pipe(

switchMap(({ accessToken }) => {

this.isRefreshing = false;

this.refreshSubject.next(accessToken);

return next.handle(this.addToken(req, accessToken));

}),

catchError(err => { this.auth.logout(); return throwError(() => err); })

);

}

private addToken(req: HttpRequest<any>, token: string) {

return req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });

}

}

```

Q16. How do you optimize Angular performance?

- **Lazy loading:** Feature modules load on demand — smaller initial bundle.
- **trackBy in *ngFor:** Angular tracks items by identity, not index — avoids destroying/recreating DOM nodes.
- **OnPush + async pipe:** Eliminates manual subscription management and reduces CD checks.
- **debounceTime on search:** Prevents API call on every keystroke.

```

// trackBy prevents full DOM rebuild on data refresh

trackById = (index: number, item: { id: number }) => item.id;

<li *ngFor="let patient of patients; trackBy: trackById">

```

```
{{ patient.name }}
```

```
</li>
```


Forms · Lifecycle Hooks · Angular ↔ .NET

Reactive Forms · ngOnChanges · ngOnDestroy · HttpClient

Q17. Template-driven vs Reactive Forms — which do you prefer and why?

	Template-driven	Reactive Forms
Definition lives in	HTML template	Component TypeScript class
Dynamic controls	Difficult	Easy — addControl() at runtime
Async validation	Awkward	First-class — AsyncValidatorFn
Unit testing	Requires DOM	Pure TS — no DOM needed
Complex scenarios	Hard to manage	Excellent
Preference	Simple login/contact forms	All enterprise forms

```
// Reactive Form – enterprise multi-step patient registration
```

```

this.patientForm = this.fb.group({

  personal: this.fb.group({

    name: ['', [Validators.required, Validators.minLength(2)]],

    dob: ['', Validators.required],

    email: ['', [Validators.required, Validators.email],

    [this.emailExistsValidator()]], // async validator

  }),

  contact: this.fb.group({

    phone: ['', Validators.pattern(/^[\d-]{10}$/)],

    address: [''],

  }),

});

```

```
// Add dynamic insurance fields at runtime
```

```

addInsurance() {

  (this.patientForm.get('insurances') as FormArray)

```

```
.push(this.fb.group({ provider: '', policyNumber: '' }));

}
```

Q18. Key lifecycle hooks and real-world use cases

Hook	When	Real Use
ngOnInit	After first change detection	Initial API call, form setup, route param subscription
ngOnChanges	Before ngOnInit + on every @Input change	Reload data when parent passes new ID
ngAfterViewInit	After view + children fully initialized	Initialize D3/Chart.js (needs DOM element)
ngOnDestroy	Before component destroyed	Unsubscribe, cancel timers, disconnect WebSocket

```
// Bulletproof memory-leak prevention pattern

export class PatientChartComponent implements OnInit, OnChanges, OnDestroy {

  @Input() patientId!: number;

  private destroy$ = new Subject<void>();

  ngOnInit() {

    // Poll vitals every 30 seconds

    interval(30_000).pipe(
      startWith(0),
      switchMap(() => this.api.getVitals(this.patientId)),
      takeUntil(this.destroy$) // auto-cancel on destroy
    ).subscribe(vitals => this.vitals = vitals);

  }

  ngOnChanges(changes: SimpleChanges) {

    // Doctor switches patient in dropdown

    if (changes['patientId'] && !changes['patientId'].firstChange) {

      this.loadPatientHistory(this.patientId);

    }

  }

}
```

```
}

ngOnDestroy() {

  this.destroy$.next();

  this.destroy$.complete(); // cancel all takeUntil subscriptions

}

}
```

■ **Real-world scenario:** Dashboard with 10 live polling APIs (vitals, alerts, queue lengths). Without `takeUntil(destroy$)`: after 2 hours the browser tab used 1.2 GB RAM — subscriptions kept firing for components that no longer existed in the DOM. Pattern above fixed it in 15 minutes.

SQL Server – Table Design & Indexing

Normalization · Primary Keys · Covering Indexes · Partitioning

Q21. How do you design tables and indexing strategies for high-transaction systems?

Good schema design is the foundation everything else rests on. Get this wrong and no amount of query tuning will save you. The principles: normalize to reduce anomalies, choose clustered index carefully (it's the physical sort order), add non-clustered covering indexes for your most frequent queries, and partition for archival patterns.

```
-- Well-designed schema example

CREATE TABLE Patients (

    PatientId INT IDENTITY(1,1) PRIMARY KEY, -- clustered by default

    HospitalId INT NOT NULL REFERENCES Hospitals(HospitalId),

    FirstName NVARCHAR(50) NOT NULL,

    LastName NVARCHAR(50) NOT NULL,

    DateOfBirth DATE NOT NULL,

    Email NVARCHAR(100) NULL,

    IsActive BIT NOT NULL DEFAULT 1,

    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()

);

-- Non-clustered index for frequent filter: active patients by hospital

CREATE NONCLUSTERED INDEX IX_Patients_Hospital_Active

ON Patients (HospitalId, IsActive)

INCLUDE (FirstName, LastName, DateOfBirth); -- covering: no key lookup needed

-- Claims table partitioned by year (archival pattern)

CREATE PARTITION FUNCTION pf_ClaimYear (int)

AS RANGE RIGHT FOR VALUES (2022, 2023, 2024, 2025);

CREATE PARTITION SCHEME ps_ClaimYear
```

```
AS PARTITION pf_ClaimYear ALL TO ([PRIMARY]);
```

```
CREATE TABLE Claims (
```

```
ClaimId INT IDENTITY(1,1),
```

```
ClaimYear AS YEAR(ClaimDate) PERSISTED,
```

```
ClaimDate DATE NOT NULL,
```

```
Amount DECIMAL(18,2) NOT NULL,
```

```
PatientId INT NOT NULL,
```

```
...
```

```
) ON ps_ClaimYear (ClaimYear);
```

■ **Real-world scenario:** Claims archive: 300M rows. Queries without partitioning scanned 300M rows. After partitioning by year: `WHERE ClaimYear = 2024` scans only 40M rows. Query time: 45 seconds → 3 seconds.

DOMAIN 22

Clustered vs Non-clustered Indexes

B-tree Structure · Key Lookups · Covering Indexes

Q22. What is the difference between clustered and non-clustered indexes? Real example?

	Clustered Index	Non-clustered Index
Data storage	IS the table — rows stored in index order	Separate B-tree with row pointers (RIDs or cluster key)
Count per table	1 only	Up to 999
Default	Primary Key	Created explicitly
Best for	Range queries, ORDER BY on indexed column	Point lookups, specific column filters
Key lookup	N/A — data is here	May need key lookup back to clustered index (expensive)

```
-- Covering index: INCLUDE avoids key lookup
```

```
-- Query: SELECT FirstName, LastName FROM Patients WHERE Email = @email
```

```
-- Without INCLUDE: index seek (fast) + key lookup per row (slow at scale)
```

```
CREATE NONCLUSTERED INDEX IX_Patients_Email
```

```

ON Patients (Email);

-- With INCLUDE: index seek + data all in index leaf = no key lookup

CREATE NONCLUSTERED INDEX IX_Patients_Email_Covering

ON Patients (Email)

INCLUDE (FirstName, LastName, IsActive);

-- Composite index — column ORDER matters!

-- Supports: WHERE UserId = x AND Status = y

-- Supports: WHERE UserId = x (leading column)

-- Does NOT support: WHERE Status = y alone (not the leading column)

CREATE NONCLUSTERED INDEX IX_Orders_User_Status

ON Orders (UserId, Status)

INCLUDE (OrderDate, Total);

```

■ **Real-world scenario:** Reporting query: *SELECT Name, DOB FROM Patients WHERE Email = @email* — was doing a full table scan (4 sec, 800K logical reads). Added non-clustered index on Email with *INCLUDE (Name, DOB)* → index seek + 0 key lookups. Query time: 4 sec → 12ms. Logical reads: 800K → 3.

SQL Optimization · Transactions · Isolation Levels

Execution Plans · Sargability · Deadlock Prevention · RCSI

Q23. How do you approach optimizing a long-running SQL query?

I follow a systematic process. First: measure (don't guess). Use SET STATISTICS IO, TIME ON to get logical reads and CPU time. Then open the Actual Execution Plan to find the most expensive operator — it's almost always a Table Scan or Key Lookup. Then fix the root cause.

```
-- Step 1: Enable statistics

SET STATISTICS IO, TIME ON;

-- Step 2: Check for non-sargable predicates (can't use indexes)

-- ■ Non-sargable: function on column

WHERE YEAR(OrderDate) = 2024 -- forces table scan

WHERE LEFT(LastName, 3) = 'Smi'

-- ■ Sargable: value on right side, column naked

WHERE OrderDate >= '2024-01-01' AND OrderDate < '2025-01-01'

WHERE LastName LIKE 'Smi%'

-- Step 3: Avoid SELECT *

-- ■

SELECT * FROM Orders o JOIN Patients p ON o.PatientId = p.PatientId

-- ■

SELECT o.Id, o.OrderDate, o.Total, p.FirstName, p.LastName

FROM Orders o JOIN Patients p ON o.PatientId = p.PatientId

WHERE o.HospitalId = @HospitalId

-- Step 4: Batch large operations

-- ■ Locks entire table, transaction log explosion
```

```

DELETE FROM AuditLogs WHERE CreatedAt < '2023-01-01'

-- ■ Batch delete: 5k rows at a time

WHILE 1 = 1 BEGIN

DELETE TOP (5000) FROM AuditLogs WHERE CreatedAt < '2023-01-01'

IF @@ROWCOUNT = 0 BREAK

WAITFOR DELAY '00:00:01' -- yield to other transactions

END

```

Q24. Explain transactions and isolation levels. How do you prevent deadlocks?

Isolation Level	Dirty Read	Non-Repeatable	Phantom	Use Case
Read Uncommitted	Yes	Yes	Yes	Analytics where speed > accuracy
Read Committed (default)	No	Yes	Yes	General OLTP
Repeatable Read	No	No	Yes	Financial read-then-write patterns
Serializable	No	No	No	Critical stock/inventory transactions
Snapshot (RCSI)	No	No	No	High-concurrency, zero blocking reads

```

-- Enable RCSI (best practice for OLTP – readers don't block writers)

ALTER DATABASE HospitalDB SET READ_COMMITTED_SNAPSHOT ON;

-- Deadlock prevention: consistent table access order

-- Always lock Accounts THEN Transactions (never reverse)

BEGIN TRAN

UPDATE Accounts SET Balance = Balance - @amount WHERE Id = @from

UPDATE Accounts SET Balance = Balance + @amount WHERE Id = @to

INSERT INTO Transactions (FromId, ToId, Amount) VALUES (@from, @to, @amount)

COMMIT

-- Retry on deadlock (C#)

```



```
for (int attempt = 0; attempt < 3; attempt++) {  
  
    try {  
  
        await ExecuteTransactionAsync();  
  
        break;  
  
    } catch (SqlException ex) when (ex.Number == 1205) { // deadlock  
  
        if (attempt == 2) throw;  
  
        await Task.Delay(100 * (attempt + 1));  
  
    }  
  
}
```

■ **Real-world scenario:** Billing service deadlock: two payment threads each locking Accounts then Billing_Ledger in different order. Thread A held Accounts, Thread B held Billing_Ledger, both waited forever. Fix: standardized order (Accounts → Billing_Ledger in all code paths) + enabled RCSI so reporting queries no longer blocked payment updates.

SQL Advanced Topics

Stored Procs · Pagination · Normalization · Migrations · DB Security · Data Modelling

Q25. How do you design stored procedures for large data processing?

```
CREATE PROCEDURE usp_ImportClaims

@BatchSize INT = 5000

AS BEGIN

SET NOCOUNT ON;

-- Stage data in temp table for faster processing

SELECT ClaimId, PatientId, Amount, ClaimDate, Status

INTO #StagedClaims

FROM ClaimStaging

WHERE ImportStatus = 'Pending';

-- Index the temp table

CREATE INDEX IX_SC_ClaimId ON #StagedClaims(ClaimId);

DECLARE @ProcessedCount INT = 0;

WHILE 1 = 1 BEGIN

BEGIN TRAN

INSERT INTO Claims (PatientId, Amount, ClaimDate, Status)

SELECT TOP (@BatchSize) PatientId, Amount, ClaimDate, Status

FROM #StagedClaims

WHERE Processed = 0;

SET @ProcessedCount = @@ROWCOUNT;

IF @ProcessedCount = 0 BEGIN COMMIT; BREAK; END
```

```

-- Mark batch as processed

UPDATE TOP (@BatchSize) #StagedClaims SET Processed = 1 WHERE Processed = 0;

COMMIT

WAITFOR DELAY '00:00:00.500'; -- yield between batches

END

DROP TABLE #StagedClaims;

END

```

Q26. How do you handle pagination for tables with millions of records?

Method	SQL	Performance on Page 10,000	Notes
OFFSET/FETCH	OFFSET n ROWS FETCH NEXT x	Slow — scans all n rows first	OK for small tables
Keyset / Seek	WHERE Id > @LastId ORDER BY Id	Constant — index seek	Best for infinite scroll

```

-- ■ OFFSET/FETCH – gets slower as page number increases

SELECT Id, PatientName, ClaimDate, Amount

FROM Claims

ORDER BY Id

OFFSET 500000 ROWS FETCH NEXT 50 ROWS ONLY; -- must skip 500k rows

-- ■ Keyset pagination – always fast, uses index

SELECT TOP 50 Id, PatientName, ClaimDate, Amount

FROM Claims

WHERE Id > @LastSeenId -- from previous page's last row

ORDER BY Id;

-- EF Core: Keyset pagination

var page = await _ctx.Claims

```

```

.AsNoTracking()

.Where(c => c.Id > lastSeenId)

.OrderBy(c => c.Id)

.Take(50)

.Select(c => new ClaimDto { ... })

.ToListAsync();

```

■ **Real-world scenario:** Admin claims grid: 12 million rows. OFFSET on page 200 took 8 seconds — had to skip 10,000 rows every time. Keyset with WHERE Id > lastId used an index seek and returned results in 40ms regardless of how deep into the data the user was.

Q27. Normalization vs Denormalization — when to use each?

	Normalization (3NF)	Denormalization
Goal	Eliminate redundancy, ensure consistency	Optimize read performance
Joins	More joins required	Fewer joins — pre-joined data
Storage	Less storage	More storage (duplicated data)
Use for	OLTP: orders, payments, patient records	OLAP: dashboards, reports, analytics

A common real-world pattern: **normalize the transactional source of truth**, then create a **denormalized read model** (materialized view or separate reporting table) for dashboards. The two are kept in sync via events or scheduled jobs.

Q28. How do you manage DB versioning and migrations in CI/CD?

```

# Create migration

dotnet ef migrations add AddClaimStatusIndex --project HCA.Infrastructure --startup-project HCA.API

dotnet ef database update

# azure-pipelines.yml – apply migrations in pipeline

- task: DotNetCoreCLI@2

  displayName: 'Apply EF Core Migrations'

  inputs:

    command: 'run'

    projects: 'src/HCA.MigrationRunner/HCA.MigrationRunner.csproj'

    env:

```

```

ConnectionStrings__DefaultConnection: $(DbConnectionString)

# Best practices:

# - Never modify existing migrations – always add new ones

# - Always write a corresponding rollback script

# - Run migrations as a K8s Job before pod rollout (init container pattern)

# - Test migrations on a copy of production data in staging first

```

Q29. How do you secure SQL Server databases?

Layer	Technique	Prevents
Application	Parameterized queries / EF Core ORM	SQL Injection
Application	Input validation + DTO mapping	Malformed / oversized inputs
Database	TDE (Transparent Data Encryption)	Data theft from backup files
Database	Always Encrypted (column level)	DBA seeing SSN, DOB, card numbers
Database	Row-Level Security	Tenant data leakage in multi-tenant systems
Database	Least privilege accounts	Lateral movement if account compromised
Audit	SQL Server Audit / DDL triggers	Compliance — who changed what, when

■ **Real-world scenario:** HCA compliance requirement: patient SSNs encrypted at column level using Always Encrypted. Even DBAs running `SELECT * FROM Patients` see ciphertext. Only the application with the correct certificate can decrypt. Azure Key Vault stores the CMK. Audit logs track every `SELECT`.

Q30. What is your data modelling approach for a new microservice?

- **1. Understand the bounded context** — what business problem does this service own? What data is it the system of record for?
- **2. Identify entities and value objects** — entities have identity (Patient, Order); value objects don't (Address, Money).
- **3. Define relationships and cardinality** — Customer → Orders (1:N), Orders ↔ Products (M:N via OrderItems).
- **4. Analyse query patterns** — read-heavy? Write-heavy? Reporting queries? This drives normalization decisions.
- **5. Plan indexing strategy upfront** — identify the top 5 most frequent queries. Design indexes for them.
- **6. Decide normalization level** — 3NF for OLTP source of truth; selectively denormalize for read models.
- **7. Plan for growth** — will this service ever have multi-tenancy? Will records be archived? Design for it now.
- **8. Database per service** — never share a database between microservices. Use events for cross-service consistency.

SOLID Principles – With Real Code

S · O · L · I · D — Senior level application

QB1. Explain SOLID principles with production-level code examples.

SOLID is not just interview theory — it's the difference between code that's easy to change and code that breaks when you touch it. The most impactful principles in practice are SRP (keeps classes focused), OCP (lets you add features without touching existing code), and DIP (makes unit testing possible).

S — SINGLE RESPONSIBILITY PRINCIPLE

```
// ■ Violates SRP: one class handles business logic + persistence + sending emails
```

```
public class OrderService {

    public void PlaceOrder(Order order) {

        ValidateOrder(order); // business logic

        _context.Orders.Add(order); // persistence

        _context.SaveChanges();

        new SmtpClient().Send(order.Email, "Confirmed"); // email

    }

}
```

```
// ■ SRP: each class has one job
```

```
public class OrderValidator { void Validate(Order o) { ... } }

public class OrderRepository { Task SaveAsync(Order o) { ... } }

public class OrderEmailSender { Task SendConfirmAsync(Order o) { ... } }

public class OrderService {

    // orchestrates, doesn't implement

    public async Task PlaceOrder(Order o) {

        _validator.Validate(o);

        await _repo.SaveAsync(o);

        await _email.SendConfirmAsync(o);

    }

}
```

```
}  
  
}
```

O — OPEN/CLOSED PRINCIPLE

```
// ■ Adding a new exporter = modify existing class = risk of regression  
  
public class ReportService {  
  
    public byte[] Export(string format) {  
  
        if (format == "PDF") return GeneratePdf();  
  
        if (format == "Excel") return GenerateExcel();  
  
        // Adding CSV means editing this method  
  
    }  
  
}
```



```
// ■ OCP: add new exporter = add new class, zero modification  
  
public interface IReportExporter { byte[] Export(ReportData data); }  
  
public class PdfExporter : IReportExporter { ... }  
  
public class ExcelExporter : IReportExporter { ... }  
  
public class CsvExporter : IReportExporter { ... } // NEW – no existing code touched  
  
// DI registration by config  
  
var exporter = config["Report:Format"] == "PDF"  
  
? (IReportExporter)new PdfExporter()  
  
: new CsvExporter();  
  
services.AddScoped(_ => exporter);
```

D — DEPENDENCY INVERSION PRINCIPLE

```
// High-level module should not depend on low-level module  
  
// Both should depend on abstractions  
  
// ■ OrderService depends on concrete SqlPatientRepository  
  
public class OrderService {
```

```
private SqlPatientRepository _repo = new SqlPatientRepository();

}

// ■ OrderService depends on IPatientRepository abstraction

public class OrderService {

    private readonly IPatientRepository _repo;

    public OrderService(IPatientRepository repo) => _repo = repo; // DI

    // In tests: inject InMemoryPatientRepository – no DB needed

}
```

■ **Real-world scenario:** HCA Billing: Report service initially only generated PDFs. Client asked for CSV and Excel. Because of OCP, we added PdfExporter, CsvExporter, ExcelExporter — zero changes to ReportService, zero regression tests failed. Adding a 4th format (Word) took one afternoon.

DOMAIN BONUS B

Design Patterns in .NET Production Code

Repository · Factory · Strategy · Decorator · CQRS

QB2. Which design patterns have you used? Scenario: 4 databases — which pattern?

I always prefer patterns that emerge from real problems rather than patterns applied for their own sake. The ones I reach for most often: Repository for data access, Strategy for swappable algorithms, Factory/Abstract Factory for object creation, and CQRS+MediatR as the backbone of Clean Architecture.

Pattern	Purpose	Where I Used It
Repository + UoW	Abstract data access; single SaveChanges across multiple repos	All services — keeps EF out of business logic
Abstract Factory	Create family of related objects (right DbContext for 4 DBs)	Multi-DB migration project
Strategy	Swap algorithm at runtime without if-else	Payment: Stripe vs Razorpay vs PayPal
CQRS + MediatR	Separate read/write; pipeline behaviors for validation+logging	All Clean Architecture projects
Decorator	Add cross-cutting concerns transparently	Cached repository wrapping real repository
Singleton	Shared instance (thread-safe)	Redis connection multiplexer, config

STRATEGY PATTERN — PAYMENT GATEWAYS

```
// Adding a new payment provider = new class only

public interface IPaymentStrategy {

    Task<PaymentResult> ChargeAsync(decimal amount, string currency);

}

public class StripePayment : IPaymentStrategy { ... }

public class RazorpayPayment : IPaymentStrategy { ... }

public class PayPalPayment : IPaymentStrategy { ... } // new — no existing code changed

// Factory selects strategy by patient country

public class PaymentStrategyFactory {

    public IPaymentStrategy Create(string country) => country switch {

        "US" or "UK" => new StripePayment(_config),

        "IN" => new RazorpayPayment(_config),
```

```

    _ => new PayPalPayment(_config),

};

}

```

DECORATOR — CACHED REPOSITORY

```

// Original

public class PatientRepository : IPatientRepository { ... }

// Decorator adds caching transparently

public class CachedPatientRepository : IPatientRepository {

    private readonly IPatientRepository _inner;

    private readonly ICacheService _cache;

    public CachedPatientRepository(IPatientRepository inner, ICacheService cache) {

        _inner = inner; _cache = cache;

    }

    public async Task<Patient?> GetByIdAsync(int id) {

        return await _cache.GetOrSetAsync(

            $"patient:{id}",

            () => _inner.GetByIdAsync(id));

    }

}

// DI wiring: inject CachedPatientRepository when IPatientRepository is requested

services.AddScoped<PatientRepository>();

services.AddScoped<IPatientRepository>(sp =>

    new CachedPatientRepository(sp.GetRequiredService<PatientRepository>(),

        sp.GetRequiredService<ICacheService>()));

```

■ **Real-world scenario:** Multi-database migration: Patient data in SQL Server, Claims in PostgreSQL, Audit in MongoDB, Legacy in Oracle. Abstract Factory returned the correct DbContext. Service layer used IUnitOfWork — completely unaware of which DB it was talking to. When Claims moved from PostgreSQL to SQL Server: one line changed in the factory. Zero business logic changes.

DOMAIN BONUS C

ASP.NET Core Middleware Pipeline

Request Pipeline · Custom Middleware · Order Matters

QB3. Explain the middleware pipeline and how you've used custom middleware.

In ASP.NET Core, every HTTP request passes through a pipeline of middleware components. Each component can: inspect/modify the request, call the next middleware, inspect/modify the response. **Order is critical** — exception handling must be first so it can catch errors from any downstream component.

```
// Correct middleware order in Program.cs

app.UseMiddleware<ExceptionMiddleware>(); // FIRST: catches all downstream errors

app.UseHttpsRedirection();

app.UseRouting();

app.UseAuthentication(); // identity from JWT

app.UseAuthorization(); // role/policy checks

app.UseMiddleware<CorrelationIdMiddleware>();

app.UseMiddleware<RequestLoggingMiddleware>();

app.MapControllers(); // LAST: actual endpoint
```

PRODUCTION MIDDLEWARE EXAMPLES

```
// 1. Correlation ID – trace one request across all microservices

public class CorrelationIdMiddleware {

    private readonly RequestDelegate _next;

    const string HEADER = "X-Correlation-ID";

    public async Task InvokeAsync(HttpContext ctx) {

        var correlationId = ctx.Request.Headers[HEADER].FirstOrDefault()

        ?? Guid.NewGuid().ToString();

        ctx.Items[HEADER] = correlationId;

        ctx.Response.Headers.Append(HEADER, correlationId);

        using (LogContext.PushProperty("CorrelationId", correlationId))
```

```

await _next(ctx);

}

}

// 2. Global exception handler – consistent error responses

public class ExceptionMiddleware {

    private readonly RequestDelegate _next;

    private readonly ILogger _logger;

    public async Task InvokeAsync(HttpContext ctx) {

        try { await _next(ctx); }

        catch (ValidationException ex) {

            ctx.Response.StatusCode = 400;

            await ctx.Response.WriteAsJsonAsync(new {

                error = "Validation failed", details = ex.Errors

            });

        }

        catch (NotFoundException ex) {

            ctx.Response.StatusCode = 404;

            await ctx.Response.WriteAsJsonAsync(new { error = ex.Message });

        }

        catch (Exception ex) {

            _logger.LogError(ex, "Unhandled exception for {Path}", ctx.Request.Path);

            ctx.Response.StatusCode = 500;

            await ctx.Response.WriteAsJsonAsync(new { error = "Internal server error" });

        }

    }

}

```

■ **Real-world scenario:** Added *CorrelationIdMiddleware* that generated a *UUID* per request and propagated it as an *HTTP* header and a *RabbitMQ* message property. With *Serilog* enriched with *CorrelationId*, we could trace one patient appointment request through 6 microservices in *App Insights* in 30 seconds — previously that investigation took 2 hours of log grepping.

async/await – Internals & Common Pitfalls

Thread Pool · ConfigureAwait · Deadlocks · Task.WhenAll

QB4. How does async/await work under the hood? Why does .Result cause deadlocks?

async/await is a compiler transformation. When the compiler sees 'await someTask', it registers a **continuation** (the code after the await) and returns the thread to the thread pool. When the awaited operation completes, the runtime picks up the continuation (possibly on a different thread). This is how one thread can serve hundreds of concurrent requests — it's not blocked waiting for I/O.

```
// ■ DEADLOCK scenario (classic ASP.NET SynchronizationContext trap)

public IActionResult GetUser(int id) {

    // .Result blocks the current thread AND waits for the

    // async continuation to run on the SAME thread → deadlock

    var user = _service.GetUserAsync(id).Result;

    return Ok(user);

}

// ■ Always async all the way down

public async Task<IActionResult> GetUser(int id) {

    var user = await _service.GetUserAsync(id); // thread released, no deadlock

    return Ok(user);

}

// ■ Parallel async operations – don't await sequentially if independent

// ■ Sequential: 3 seconds total (1s + 1s + 1s)

var patients = await _patientRepo.GetAllAsync();

var doctors = await _doctorRepo.GetAllAsync();

var departments = await _deptRepo.GetAllAsync();

// ■ Parallel: ~1 second total
```

```

var (patients, doctors, departments) = await (

_patientRepo.GetAllAsync(),

_doctorRepo.GetAllAsync(),

_deptRepo.GetAllAsync()

).WhenAll();

// Helper extension for tuple deconstruction

var results = await Task.WhenAll(t1, t2, t3);

// ■ ConfigureAwait(false) in library code – avoids context capture

public async Task<byte[]> ReadFileAsync(string path) {

return await File.ReadAllBytesAsync(path).ConfigureAwait(false);

}

```

■ **Real-world scenario:** Patient API had `.Result` in an authorization filter that validated hospital access. In load testing at 200 concurrent users, thread pool threads exhausted — each blocked waiting for async work that needed the same threads. All requests returned 503. Fix: made the filter async (`IAsyncAuthorizationFilter`). Throughput: 50 req/s → 800 req/s. No code changes except `async` keywords.

■ **Common Mistake:** `async void` is only valid for event handlers. Use `async Task` everywhere else. `async void` exceptions crash the process silently.

DOMAIN BONUS E

Azure Cloud & Docker

CI/CD · AKS · Key Vault · Blob Storage · Durable Functions · Dockerfile

QB5. Explain your Azure deployment flow and the services you've used.

Azure Service	Purpose	How I Used It
Azure Kubernetes Service (AKS)	Container orchestration	Deployed all 6 microservices as pods
Azure Container Registry (ACR)	Private Docker image store	CI pipeline pushed images here
Azure DevOps	CI/CD pipelines	Build → Test → Docker Build → Deploy to AKS
Azure Key Vault	Secrets management	DB connection strings, JWT keys, API keys
Azure Blob Storage	Object storage	Patient X-rays, claim PDFs, report exports
Azure Service Bus	Enterprise messaging	Alternative to RabbitMQ for Azure-native
Azure Application Insights	APM + distributed tracing	Logs, metrics, request traces across services
Azure Redis Cache	Distributed cache	Shared cache across all AKS pods
Durable Functions	Stateful serverless workflows	Patient discharge workflow (multi-step, 48hr)

PRODUCTION DOCKERFILE

```
# Multi-stage build: keeps final image small

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base

WORKDIR /app

EXPOSE 80

EXPOSE 443


FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build

WORKDIR /src

# Copy csproj first – layer caching: only reinstall packages when .csproj changes

COPY ["src/HCA.PatientAPI/HCA.PatientAPI.csproj", "src/HCA.PatientAPI/"]

COPY ["src/HCA.Application/HCA.Application.csproj", "src/HCA.Application/"]

RUN dotnet restore "src/HCA.PatientAPI/HCA.PatientAPI.csproj"
```

```
COPY . .

RUN dotnet publish "src/HCA.PatientAPI/HCA.PatientAPI.csproj" \

-c Release -o /app/publish --no-restore

FROM base AS final

WORKDIR /app

# Non-root user for security

USER app

COPY --from=build /app/publish .

ENTRYPOINT ["dotnet", "HCA.PatientAPI.dll"]
```

```
# azure-pipelines.yml

trigger: [ main ]

stages:

- stage: BuildAndTest

jobs:

- job: Build

pool: { vmImage: ubuntu-latest }

steps:

- task: DotNetCoreCLI@2

inputs: { command: test, arguments: '--collect:"XPlat Code Coverage"' }

- task: Docker@2

inputs:

containerRegistry: 'hca-acr'

repository: 'patient-api'

command: 'buildAndPush'

tags: '$(Build.BuildId)'
```

```
- stage: DeployProd

dependsOn: BuildAndTest

jobs:

- deployment: AKS

environment: production

strategy:

runOnce:

deploy:

steps:

- task: KubernetesManifest@1

inputs:

action: deploy

manifests: 'k8s/patient-api.yaml'

containers: 'hca.azurecr.io/patient-api:${Build.BuildId}'
```

■ **Real-world scenario:** *PatientService required a specific version of wkhtmltopdf for PDF generation. Dev machines had 3 different OS versions — 'it works on my machine' was constant. Dockerized the service: pinned exact Linux base image + wkhtmltopdf version. Same Docker image ran on dev, staging, and AKS production. PDF rendering issues in production: eliminated permanently.*

MCQ Cheat Sheet – Common Screening Questions

Fullstack · JavaScript · Angular · React · General Concepts

Question	Correct Answer	Why / Trick
SQL stands for	Structured Query Language	Not 'Simple'
CRUD stands for	Create, Read, Update, Delete	Standard data operations
SPA stands for	Single Page Application	Angular, React are SPAs
DOM stands for	Document Object Model	Tree representation of HTML
API stands for	Application Programming Interface	Contract between systems
PHP stands for	PHP: Hypertext Preprocessor	Recursive acronym
NPM true statement	Node Package Manager, uses package.json	Not 'Node Project Manager'
Git is	Distributed version control system	Not 'Centralized' (that's SVN)
3 states of file in Git	Modified → Staged → Committed	Not 4 — Working/Index/Repo
Virtual DOM (React)	In-memory DOM diff → minimal real DOM updates	Makes React fast
3 React lifecycle phases	Mounting → Updating → Unmounting	componentDidMount = Mounting
Django architecture	MVT: Model-View-Template	Not MVC — Template not Controller
JavaScript execution	Single-threaded, async via event loop	setTimeout is async
Template literals use	Backticks `` with \${variable}	Not single/double quotes
JS pop-up types	alert(), confirm(), prompt()	3 types — confirm returns bool
push() in JS	Adds element to END of array	pop() removes from end
Props in React	Read-only data passed parent → child	Cannot be mutated
Add JS to HTML		Not
HTML vector tag		Not (that's raster)
iframe used for	Embed another HTML page inside	Can embed YouTube, maps
Frontend frameworks	Angular, React, Vue	Bootstrap is a CSS library, not a framework
NOT a frontend tech	Node.js, SQL Server, Django	Node.js is server-side
NoSQL databases	MongoDB, Redis, Cassandra	SQL Server, MySQL are RELATIONAL
HTTP methods (REST)	9 total; main 5: GET POST PUT DELETE PATCH	HEAD, OPTIONS, CONNECT, TRACE are the rest
REST characteristics	Stateless, HTTP, resource-based	No persistent connection like WebSocket
Install Express module	npm install express	Not 'npm add express' (that's Yarn)
Scalability means	Handle increased load by adding resources	Horizontal = more servers, Vertical = bigger server

Question	Correct Answer	Why / Trick
Deployment means	Moving code from dev to production	CI/CD automates this
Python backend	Django, Flask, FastAPI	Not Ruby on Rails (that's Ruby)
MVC: View's role	Display data (UI only) — no business logic	Controller has logic
class vs id in HTML	class = reusable multiple elements; id = unique	Common mistake: thinking class is unique
AOT compilation (Angular)	Pre-compiles templates at build time	Smaller bundle, faster startup
WebSocket vs HTTP	WebSocket: persistent bidirectional; HTTP: stateless request-response	SignalR uses WebSocket
Lazy loading benefit	Reduces initial bundle size — module loads on demand	Improves first load time

Senior-Level Interview Strategy & Tips

How to answer · What interviewers really want to hear

The 5-Layer Answer Framework

Layer 1: Architecture	Why before How. 'I use X because Y.' Mention trade-offs.
Layer 2: Implementation	Code approach at a high level — don't recite syntax, explain decisions.
Layer 3: Optimization	What did you improve? Numbers matter: '8 sec → 400ms', 'DB CPU 90% → 15%'.
Layer 4: Real-world example	Always tie to a named project. HCA, healthcare, fintech — specifics build credibility.
Layer 5: Lesson learned	What would you do differently? Shows maturity and self-awareness.

Hyderabad Market — Most Frequently Asked Topics (2025–2026)

■ Microservices + RabbitMQ + Redis	<i>Every senior .NET role. Architecture Q always comes first.</i>
■ EF Core Optimisation + SQL Indexing	<i>Performance differentiation — mid vs senior. Quote numbers.</i>
■ Angular OnPush + RxJS + NgRx	<i>Heavy Angular roles. Memory leaks are the trap question.</i>
■ JWT end-to-end + Refresh Token flow	<i>HCA and fintech. Know the interceptor pattern cold.</i>
■ Clean Architecture + CQRS + MediatR	<i>Ask in most senior architect rounds. Know all 4 layers.</i>
■ async/await + .Result deadlock	<i>Classic senior trap. ConfigureAwait(false) bonus points.</i>
■ Docker + AKS + Azure DevOps pipeline	<i>Almost every role. Show you've written a Dockerfile.</i>
■ SOLID with real examples	<i>OCP and DIP are the most asked. Have code ready.</i>

Prepared for .NET Fullstack Senior Roles · ASP.NET Core · Angular · SQL Server · Azure · 2025–2026
Covers 30 Domains · 45+ Questions · Production Code · Real Scenarios · MCQ Cheat Sheet